

Ir Além 1 | Extração com Gemini

Proposta e referência

A extensão organiza textos clínicos fictícios em JSON, usando prompting e API de modelo comercial, conforme PCV, capítulo 10, pp. 57-63. Foi escolhido texto, alternativa aceita pelo enunciado; não há análise de imagens nesta implementação. O provedor solicitado é Google AI Studio e o identificador configurado é gemini-3.5-flash-lite.

Fluxo completo

Texto fictício > validação HTTP > busca no cache > reserva de tentativa no orçamento > API Gemini com instruções e esquema > validação Pydantic e evidência literal > apresentação de fatos e JSON > revisão humana > inclusão opcional do relato no contexto Watson. Sem conversa ativa, a extração continua disponível, mas não há inclusão automática no chat.

Contrato estruturado

Cada fato possui campo, valor, evidencia e estado. Os campos permitidos são sintoma, pressao_arterial, frequencia_cardiaca, inicio, historico e adesao. Estado aceita afirmado, negado ou incerto. O esquema rejeita campos adicionais, valores vazios e mais de 30 fatos. Valor deve estar dentro da evidência, e a evidência deve estar no texto de origem. A resposta inclui modelo, texto original e indicação de revisão necessária.

Prompt e proteção da fidelidade

extensions/generative/prompt.txt instrui a extrair somente declarações explícitas, conservar negações e incerteza, não resolver conflitos por suposição e ignorar instruções contidas no próprio relato. O backend também faz uma triagem conservadora de negações. Essas medidas não detectam todos os erros semânticos: o modelo ainda pode omitir fatos ou selecionar evidências incompletas.

Exemplo executado na API real

Entrada: Minha frequência foi 72 bpm. Não tenho dor. Uma resposta esperada contém frequência 72 bpm afirmada e dor negada, cada uma com trecho literal correspondente. A aplicação executou o exemplo com sucesso na API real. As respostas estão registradas em docs/evidence/gemini-evaluation.json; o exemplo foi reavaliado a partir do cache dessa chamada.

Cota, cache e erros

O orçamento local limita app e robô a até 500 tentativas nas últimas 24 horas, com intervalo mínimo padrão de 10 segundos. A reserva usa transação SQLite para concorrência e persiste após recriação dos containers. O cache usa hash do modelo, prompt e texto, com validade de 24 horas. Não há retries automáticos: falhas consomem uma tentativa. A cota do Google pode ser diferente e inclui o uso de outras aplicações.

Integração e execução

A biblioteca google-genai faz a chamada com `response_mime_type application/json` e esquema Pydantic. A interface exibe valores, estado, evidências e JSON bruto estruturado. O comando `python -m extensions.generative.extract_clinical --text` seguido do relato executa a mesma lógica no container. A confirmação inclui o texto original no Watson para conferência; não converte automaticamente uma inferência do modelo em medição clínica.

Validação e avaliação real

Os testes locais cobrem formato, evidência inventada, negação, chave ausente, cache, persistência e concorrência do orçamento. São verificações do código com respostas controladas. A avaliação real incluiu seis casos: negação, ausência de medição, duas medidas, texto sem fatos, instrução embutida e incerteza. As seis saídas passaram esquema/evidências e os seis fatos da referência foram encontrados. Houve um fato adicional de início sustentado pelo texto, omitido pela referência e revisado como válido. Não se generalizam esses resultados para outros relatos; variação entre gerações não foi medida.

Reprodução e evidências

Defina `GEMINI_API_KEY` no `.env` e recrie os containers. Use a aba Organizar relato ou o comando documentado no README. Consulte `/api/status` para tentativas locais. Nunca registre chaves junto das evidências. `docs/VALIDACAO.md` separa resultados reais de exemplos esperados e deve acompanhar este relatório.

Referências

PCV, capítulo 10, pp. 57-60: APIs de modelos comerciais; pp. 60-63: integração híbrida. Documentação do modelo: ai.google.dev/gemini-api/docs/models/gemini-3.5-flash-lite. SDK: googleapis.github.io/python-genai/. Código: `services/gemini_service.py`, `services/storage.py` e `extensions/generative/`.